INSTITUTE OF AERONAUTICAL ENGINEERING
(Autonomous)
Dundigal - 500 043, Hyderabad, Telangana

DEPARTMENT OF CSE(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

DATABASE MANAGEMENT SYSTEMS LABORATORY

Name of the Staff :Dr.B.Venkateswara Rao **Emp Id:11207**
EXERCISE –5: PROCEDURES AND FUNCTIONS **Date:27/08/2026**

1. Write a function to accept employee number as parameter and return Basic +HRA together as a single column.

Aim

To write and execute a **PL/SQL function** that accepts an employee number as a parameter and returns the sum of **Basic Salary + HRA** for that employee.

Algorithm

1. Start the PL/SQL program.
2. Create a function named GET_BASIC_HRA.
3. Accept EMPLOYEE NUMBER as an input parameter.
4. Declare a variable to store BASIC + HRA.
5. Retrieve BASIC and HRA from the EMPLOYEE table using the given employee number.
6. Calculate BASIC + HRA.
7. Store the result in the local variable.
8. Return the calculated value from the function.
9. Call the function using a SELECT statement.
10. Display the result.
11. Stop.

Table Structure

```
CREATE TABLE EMPLOYEE (
   EMPNO      NUMBER(5) PRIMARY KEY,
   ENAME      VARCHAR2(30),
   BASIC      NUMBER(10,2),
   HRA        NUMBER(10,2),
   NETSALARY  NUMBER(10,2),
   SALDATE    DATE
);
```

Sample Records

```
INSERT INTO EMPLOYEE VALUES
(101, 'RAMESH', 25000, 5000, 28000, TO_DATE('15-01-2024','DD-MM-YYYY'));

INSERT INTO EMPLOYEE VALUES
(102, 'SURESH', 30000, 6000, 34000, TO_DATE('20-03-2024','DD-MM-YYYY'));
```

```
INSERT INTO EMPLOYEE VALUES
(103, 'RAJESH', 35000, 7000, 40000, TO_DATE('10-06-2024','DD-MM-YYYY'));

INSERT INTO EMPLOYEE VALUES
(104, 'KIRAN', 28000, 5500, 31500, TO_DATE('15-02-2025','DD-MM-YYYY'));

INSERT INTO EMPLOYEE VALUES
(105, 'ANIL', 40000, 8000, 45000, TO_DATE('25-04-2025','DD-MM-YYYY'));

INSERT INTO EMPLOYEE VALUES
(106, 'PRIYA', 32000, 6500, 36500, TO_DATE('10-07-2025','DD-MM-YYYY'));

INSERT INTO EMPLOYEE VALUES
(107, 'SWATHI', 45000, 9000, 50000, TO_DATE('05-01-2026','DD-MM-YYYY'));

INSERT INTO EMPLOYEE VALUES
(108, 'KUMAR', 38000, 7500, 42500, TO_DATE('18-03-2026','DD-MM-YYYY'));

COMMIT;
```

PL/SQL Program

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION get_basic_hra(p_empno NUMBER)
RETURN NUMBER
IS
   v_total NUMBER;
BEGIN
   SELECT basic + hra
   INTO v_total
   FROM employee
   WHERE empno = p_empno;

   RETURN v_total;
END;
/
```

Execute the Function

```
SELECT get_basic_hra(101) AS "BASIC + HRA"
FROM dual;
```

Output

```
BASIC + HRA
-----------
30000
```

Using PL/SQL Block to Display Result

```
DECLARE
   v_result NUMBER;
BEGIN
   v_result := get_basic_hra(101);

   DBMS_OUTPUT.PUT_LINE('Basic + HRA = ' || v_result);
```

```
END;
/
```

Output:

```
Basic + HRA = 30000
```

---

2. Accept year as parameter and write a Function to return the total net salary spent for a given year.

Aim

To write and execute a **PL/SQL function** that accepts a year as a parameter and returns the **total net salary spent** for all employees in that particular year.

Algorithm

1. Start the PL/SQL program.
2. Create a function named TOTAL_SALARY.
3. Accept the **year** as an input parameter.
4. Declare a variable V_TOTAL to store the total net salary.
5. Retrieve the NETSALARY values from the EMPLOYEE table for the given year.
6. Calculate the total using the SUM() function.
7. Store the result in V_TOTAL.
8. Return V_TOTAL.
9. Call the function by passing the required year.
10. Display the total net salary.
11. Stop.

PL/SQL Function

```
CREATE OR REPLACE FUNCTION total_salary(p_year NUMBER)
RETURN NUMBER
IS
   v_total NUMBER;
BEGIN
   SELECT NVL(SUM(netsalary), 0)
   INTO v_total
   FROM employee
   WHERE EXTRACT(YEAR FROM saldate) = p_year;

   RETURN v_total;
END;
/
```

Check for Errors

After creating the function:

SHOW ERRORS FUNCTION total_salary;

It should display:

No errors.

Execute the Function

```
SELECT total_salary(2025) AS "TOTAL NET SALARY"
FROM dual;
```

Calculation

For the year **2025**:

```
KIRAN  = 31,500
ANIL   = 45,000
PRIYA  = 36,500
----------------
Total  = 1,13,000
```

Output

```
TOTAL NET SALARY
----------------
113000
```

---

3. Create a function to find the factorial of a given number and hence find NCR.

Aim

To write and execute a **PL/SQL function** to find the factorial of a given number and use the factorial function to calculate **NCR (n choose r)**.

Formula:

$$NCR = \frac{N!}{R!(N-R)!}$$

For example:

$$^5C_2 = \frac{5!}{2!(5-2)!} = 10$$

Algorithm

1. Start the PL/SQL program.
2. Create a function named FACTORIAL.

3. Accept a number N as a parameter.
4. Initialize FACT to 1.
5. Use a FOR LOOP from 1 to N.
6. Calculate the factorial by multiplying each number with FACT.
7. Return the factorial value.
8. Declare values for N and R.
9. Calculate NCR using the formula:
   N! / (R! × (N-R)!).
10. Display the NCR value.
11. Stop.

PL/SQL Program

Step 1: Create Factorial Function

```
CREATE OR REPLACE FUNCTION factorial(n NUMBER)
RETURN NUMBER
IS
   fact NUMBER := 1;
BEGIN
   FOR i IN 1..n LOOP
      fact := fact * i;
   END LOOP;

   RETURN fact;
END;
/
```

Step 2: Calculate NCR

```
SET SERVEROUTPUT ON;

DECLARE
   n NUMBER := 5;
   r NUMBER := 2;
   result NUMBER;
BEGIN
   result := factorial(n) /
          (factorial(r) * factorial(n-r));

   DBMS_OUTPUT.PUT_LINE('N = ' || n);
   DBMS_OUTPUT.PUT_LINE('R = ' || r);
   DBMS_OUTPUT.PUT_LINE('NCR = ' || result);
END;
/
```

Output

```
N = 5
R = 2
NCR = 10
```

Direct SQL Method

You can also test the factorial function separately:

```
SELECT factorial(5) AS FACTORIAL
FROM dual;
```

Output:

```
FACTORIAL
---------
120
```

And NCR:

```
SELECT factorial(5) /
     (factorial(2) * factorial(3)) AS NCR
FROM dual;
```

Output:

```
NCR
---
10
```

Important Note

For NCR, the condition should be:

```
N >= R
```

For example:

```
5C2  = 10
5C3  = 10
6C2 = 15
10C2 = 45
```

---

4) Write a PL/SQL block to print prime Fibonacci series using local functions.

Aim

To write and execute a **PL/SQL block using a local function** to generate Fibonacci numbers and print only the **prime numbers** from the Fibonacci series.

Algorithm

1. Start the PL/SQL block.
2. Initialize the first two Fibonacci numbers as 0 and 1.
3. Define a local function IS_PRIME to check whether a number is prime.
4. Generate Fibonacci numbers using a FOR LOOP.
5. For each Fibonacci number, call the IS_PRIME function.
6. If the number is prime, print it.
7. Generate the next Fibonacci number by adding the previous two numbers.
8. Repeat for the required number of terms.
9. Stop.

PL/SQL Program

```
SET SERVEROUTPUT ON;
```

```
DECLARE
   a NUMBER := 0;
   b NUMBER := 1;
   c NUMBER;

   -- Local function to check prime number
   FUNCTION is_prime(n NUMBER)
   RETURN BOOLEAN
   IS
   BEGIN
      IF n < 2 THEN
         RETURN FALSE;
      END IF;

      FOR i IN 2..TRUNC(SQRT(n)) LOOP
         IF MOD(n, i) = 0 THEN
            RETURN FALSE;
         END IF;
      END LOOP;

      RETURN TRUE;
   END;

BEGIN
   DBMS_OUTPUT.PUT_LINE('Prime Fibonacci Numbers:');

   FOR i IN 1..10 LOOP

      IF is_prime(a) THEN
         DBMS_OUTPUT.PUT_LINE(a);
      END IF;

      c := a + b;
      a := b;
      b := c;

   END LOOP;
END;
/
```

Output

For the first 10 Fibonacci terms:
Prime Fibonacci Numbers:
2
3
5
13

==========Write up to Here==========

Fibonacci Series

The first Fibonacci numbers are:
0 1 1 2 3 5 8 13 21 34
The prime numbers among them are:
2 3 5 13

Important Point

The function
FUNCTION is_prime(n NUMBER)
RETURN BOOLEAN
is a **local function**, because it is declared inside the DECLARE section of the PL/SQL block.

5) Create a procedure to find the lucky number of a given birth date.

Aim
To write and execute a PL/SQL procedure to find the lucky number of a given birth date by adding all the digits of the date repeatedly until a single digit is obtained.
Example
For birth date 12-06-1989:
1 + 2 + 0 + 6 + 1 + 9 + 8 + 9 = 36
3 + 6 = 9
Therefore, the Lucky Number = 9.
Algorithm

1. **Start the PL/SQL program.**
2. **Create a procedure named LUCKY_NUMBER.**
3. **Accept the date of birth as a parameter.**
4. **Convert the date into DDMMYYYY format.**
5. **Extract each digit from the date.**
6. **Add all the digits.**
7. **If the result contains more than one digit, repeat the process.**
8. **Continue until a single digit is obtained.**
9. **Display the single digit as the lucky number.**
10. **Stop.**

PL/SQL Program

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE PROCEDURE lucky_number(p_dob DATE)
IS
   v_num  NUMBER;
   v_sum  NUMBER;
BEGIN
   -- Convert date into DDMMYYYY
   v_num := TO_NUMBER(TO_CHAR(p_dob, 'DDMMYYYY'));

   -- Repeat until a single digit is obtained
   WHILE v_num >= 10 LOOP
      v_sum := 0;

      WHILE v_num > 0 LOOP
         v_sum := v_sum + MOD(v_num, 10);
         v_num := TRUNC(v_num / 10);
      END LOOP;

      v_num := v_sum;
   END LOOP;

   DBMS_OUTPUT.PUT_LINE('Lucky Number = ' || v_num);
END;
/
```

<u>Execute the Procedure:</u>

```
BEGIN
   lucky_number(TO_DATE('12-06-1989', 'DD-MM-YYYY'));
END;
/
```

Output
Lucky Number = 9

Another Example:
For birth date 29-05-2011:
2 + 9 + 0 + 5 + 2 + 0 + 1 + 1 = 20
2 + 0 = 2
Execute:

```
BEGIN
   lucky_number(TO_DATE('29-05-2011', 'DD-MM-YYYY'));
END;
/
```

Output:
Lucky Number = 2
Important PL/SQL Functions Used
TO_CHAR(p_dob, 'DDMMYYYY')
Converts the date into the required digit format.

MOD(v_num, 10)
Extracts the last digit.

TRUNC(v_num / 10)
Removes the last digit.

---

6) Create a function to reverse a given number.

Aim
To write and execute a **PL/SQL function** to find the reverse of a given number.
Algorithm

1. Start the PL/SQL program.
2. Create a function named REVERSE_NUMBER.
3. Accept a number as a parameter.
4. Initialize REV to 0.
5. Extract the last digit using MOD(number, 10).
6. Add the extracted digit to REV.
7. Remove the last digit using TRUNC(number / 10).
8. Repeat the process using a WHILE LOOP until the number becomes 0.
9. Return the reversed number.
10. Display the result.
11. Stop.

PL/SQL Program

```
SET SERVEROUTPUT ON;

CREATE OR REPLACE FUNCTION reverse_number(p_num NUMBER)
RETURN NUMBER
IS
   v_num   NUMBER := p_num;
   v_rev   NUMBER := 0;
   v_digit NUMBER;
BEGIN
   WHILE v_num > 0 LOOP

      -- Extract last digit
      v_digit := MOD(v_num, 10);

      -- Add digit to reverse number
      v_rev := v_rev * 10 + v_digit;

      -- Remove last digit
      v_num := TRUNC(v_num / 10);
```

```
    END LOOP;

    RETURN v_rev;
END;
/
```

Execute the Function

```
SELECT reverse_number(12345) AS "REVERSE NUMBER"
FROM dual;
```

Output

```
REVERSE NUMBER
--------------
54321
```

Using PL/SQL Block

```
DECLARE
    v_result NUMBER;
BEGIN
    v_result := reverse_number(12345);

    DBMS_OUTPUT.PUT_LINE('Original Number = 12345');
    DBMS_OUTPUT.PUT_LINE('Reverse Number = ' || v_result);
END;
/
```

Output

```
Original Number = 12345
Reverse Number = 54321
```

Logic Example

For 12345:

Last digit = 5 → Reverse = 5
Last digit = 4 → Reverse = 54
Last digit = 3 → Reverse = 543
Last digit = 2 → Reverse = 5432
Last digit = 1 → Reverse = 54321

Functions used:

MOD(n,10) → gets the last digit.
TRUNC(n/10) → removes the last digit.